

Determinar un método para migrar la base de datos desde el entorno relacional a una base de datos NoSQL

Para realizar la migración del modelo relacional de "GC Distribuidor" hacia un entorno NoSQL (específicamente orientado a documentos como MongoDB), se propone un enfoque de Desnormalización y Proceso ETL (Extracción, Transformación y Carga) basado en archivos intermedios JSON, estructurado en tres fases:

- **Reestructuración del Modelo (Desnormalización):** El paradigma relacional fragmenta la información para evitar redundancias (Ej. separar Ventas, Clientes y Detalle_Ventas). En el modelo NoSQL, el objetivo es favorecer la velocidad de lectura. Por lo tanto, se rediseña el modelo agrupando los datos relacionados en Documentos BSON únicos y jerárquicos.

Por ejemplo, la colección ventas ahora contendrá la cabecera de la factura, los datos esenciales del cliente como un subdocumento incrustado, y un arreglo (array) con todos los productos de esa transacción.

- **Extracción y Transformación (Capa Relacional):** Utilizando cualquier cliente de administración de bases de datos estándar (como Oracle SQL Developer, Toad o herramientas genéricas), se construyen consultas SQL (usando JOIN) que unifican las tablas relacionales para que coincidan con la nueva estructura jerárquica planificada.

El conjunto de resultados de estas consultas se exporta a un formato de texto estructurado y universal, preferiblemente JSON.

- **Carga de Datos (Capa NoSQL):** Con los archivos JSON generados listos, se procede a poblar la nueva base de datos NoSQL utilizando las herramientas de importación masiva nativas del motor seleccionado (por ejemplo, ejecutando el comando estándar mongoimport).

Este proceso lee los archivos locales y genera automáticamente las colecciones (clientes, productos, ventas), dejando la base de datos operativa y lista para recibir consultas mediante su Aggregation Framework.

Consultas Complejas (Oracle SQL vs MongoDB)

- Total de ingresos generados por cada cliente (Ordenado de mayor a menor)

SQL:

The screenshot shows the Oracle SQL Developer interface. The main window displays a SQL script with the following query:

```
1 SELECT
2     c.nombres_apellidos AS cliente,
3     SUM(v.valor_total) AS total_gastado
4 FROM CLIENTES c
5 JOIN VENTAS v ON c.codigo_cliente = v.codigo_cliente
6 GROUP BY c.nombres_apellidos
7 ORDER BY total_gastado DESC;
```

Below the script, the results are displayed in a grid format:

	AZ CLIENTE	123 TOTAL_GASTADO
1	Maria Gomez	630
2	Juan Jose Benitez Castro	440
3	Pedro Pascal	411
4	Ana Torres	270

A status dialog box is open on the right, displaying the message: "4 row(s) fetched - 0.003s, on 2026-07-20 at 12:07:31".

Obliga a usar JOIN para cruzar las tablas CLIENTES y VENTAS. Luego, usa GROUP BY para agrupar por cliente y SUM para el total.

NoSQL:

```
gcdistribuidor> db.ventas.aggregate([
|   {
|     $group: {
|       _id: "$cliente.nombres_apellidos",
|       total_gastado: { $sum: "$valor_total" }
|     }
|   },
|   {
|     $sort: { total_gastado: -1 }
|   }
| ]);
|
| [
|   { _id: 'Maria Gomez', total_gastado: 630 },
|   { _id: 'Juan Jose Benitez Castro', total_gastado: 440 },
|   { _id: 'Pedro Pascal', total_gastado: 411 },
|   { _id: 'Ana Torres', total_gastado: 270 }
| ]
```

Como los datos del cliente están incrustados, nos ahorramos el join. Con el Aggregation Framework, usamos \$group para leer directamente el subdocumento cliente.nombres_apellidos y \$sum para acumular el valor.

- **Top 5 Productos más vendidos por cantidad de unidades**

SQL:

```

1 SELECT
2   p.nombre,
3   SUM(dv.cantidad) AS total_unidades_vendidas
4 FROM PRODUCTOS p
5 JOIN DETALLE_VENTAS dv ON p.codigo_producto = dv.codigo_producto
6 GROUP BY p.nombre
7 ORDER BY total_unidades_vendidas DESC
8 FETCH FIRST 5 ROWS ONLY;

```

AZ NOMBRE	TOTAL_UNIDADES_VENDIDAS
1 Zocalo MDF	80
2 Espuma Aislante 2mm	50
3 Piso Laminado Roble	20
4 Piso PVC Gris	20
5 Piso Flotante Aleman 8mm	10

Message: 5 row(s) fetched - 0.005s, on 2026-07-20 at 12:10:49

Cruza las tablas, suma las cantidades vendidas, ordena y utiliza FETCH FIRST 5 ROWS ONLY (paginación de Oracle) para obtener el Top 5.

NoSQL:

```

gcdistribuidor> db.ventas.aggregate([
|   { $unwind: "$detalles" },
|   {
|     $group: {
|       _id: "$detalles.nombre_producto",
|       total_unidades_vendidas: { $sum: "$detalles.cantidad" }
|     }
|   },
|   { $sort: { total_unidades_vendidas: -1 } },
|   { $limit: 5 }
| ]);
|
| [
|   { _id: 'Zocalo MDF', total_unidades_vendidas: 80 },
|   { _id: 'Espuma Aislante 2mm', total_unidades_vendidas: 50 },
|   { _id: 'Piso PVC Gris', total_unidades_vendidas: 20 },
|   { _id: 'Piso Laminado Roble', total_unidades_vendidas: 20 },
|   { _id: 'Piso Flotante Aleman 8mm', total_unidades_vendidas: 10 }
| ]

```

Dado que los productos están dentro de un array (detalles), es obligatorio usar primero la etapa \$unwind para "desempaquetarlos". Recién ahí se pueden agrupar, ordenar y acortar con \$limit: 5

- **Alerta de Stock Crítico (Productos cuyo stock disponible cruzó el límite mínimo)**

SQL:

```

1 SELECT
2     p.nombre AS producto,
3     i.stock_disponible,
4     i.stock_minimo
5 FROM PRODUCTOS p
6 JOIN INVENTARIO i ON p.codigo_producto = i.codigo_producto
7 WHERE i.stock_disponible <= i.stock_minimo;

```

PRODUCTOS(+) 1 X

SELECT p.nombre, SUM(dv.cantidad) AS total

Grid	Az PRODUCTO	123 STOCK_DISPONIBLE	123 STOCK_MINIMO
1	Barredera PVC	8	10
2	Piso Laminado Roble	10	15
3	Perfil Transicion	9	10

Message: 3 row(s) fetched - 0.005s, on 2026-07-20 at 12:12:44

Tras unir tablas, usa un simple condicional WHERE para comparar fila por fila si stock_disponible <= stock_minimo.

NoSQL:

```

gcdistribuidor> db.productos.find(
| {
|   $expr: {
|     $lte: [ "$inventario.stock_disponible", "$inventario.stock_minimo" ]
|   }
| },
| {
|   nombre: 1,
|   "inventario.stock_disponible": 1,
|   "inventario.stock_minimo": 1,
|   _id: 0
| }
| );
| [
|   {
|     nombre: 'Barredera PVC',
|     inventario: { stock_minimo: 10, stock_disponible: 8 }
|   },
|   {
|     nombre: 'Piso Laminado Roble',
|     inventario: { stock_minimo: 15, stock_disponible: 10 }
|   },
|   {
|     nombre: 'Perfil Transicion',
|     inventario: { stock_minimo: 10, stock_disponible: 9 }
|   }
| ]

```

Usa el método .find(), pero requiere obligatoriamente el operador avanzado \$expr con \$lte para poder comparar matemáticamente dos campos que están dentro del mismo documento BSON. Finalmente, usa una proyección (1 y 0) para filtrar visualmente los datos y ocultar el _id.